

Indexing & Query Performance

Part III · Data & Storage — turning slow scans into fast lookups.

LEARNING OBJECTIVES

- Explain why an un-indexed query does a full table scan, and why that's slow.
- Explain what an index is and how a B-tree makes lookups fast.
- Know what indexes speed up — and what they cost.
- Distinguish clustered, composite, and covering indexes.
- Read a query plan ([EXPLAIN](#)) to see what the database actually does.
- Fix common query-performance problems beyond indexing (like the N+1 problem).

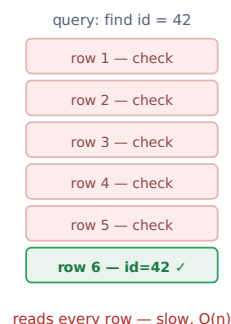
REAL-WORLD ANALOGY — THE INDEX AT THE BACK OF A BOOK

To find every mention of “caching” in a 900-page book, you could read all 900 pages — or you could flip to the **index** at the back, find “caching,” and jump straight to the listed pages. A database **index** is exactly that: a small, sorted lookup structure that points to the rows you want, so the database never has to read the whole table.

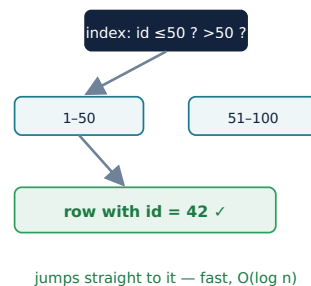
12.1 The problem: full table scans

From Chapter 11, a query like `WHERE user_id = 42` must find matching rows. With no help, the database performs a **full table scan** — it reads every row to check the condition. That's fine for a hundred rows, but a table with ten million rows means ten million reads for one lookup — and, recalling Chapter 2, touching disk that much is slow. As data grows, un-indexed queries go from instant to unusable.

WITHOUT INDEX — full scan



WITH INDEX — seek

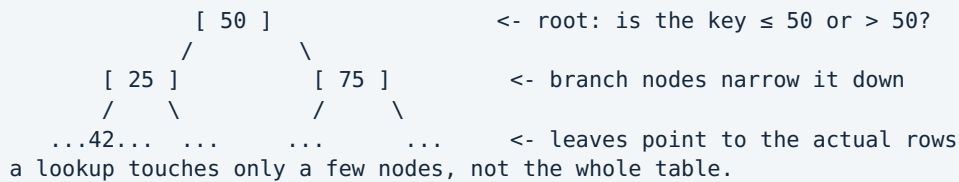


12.2 What an index is

An **index** is a separate, sorted data structure that maps column values to the locations of the matching rows. Instead of scanning the table, the database consults the index — already sorted — and jumps to the rows it needs. Like the book's index, it's extra pages that make finding things fast.

12.3 How indexes work: the B-tree

The default index structure is a **B-tree** (balanced tree). Keys are kept sorted in a shallow, wide tree; to find a value the database walks from the root down to a leaf, comparing at each level. Because the tree is balanced and shallow, a lookup among millions of rows takes only a handful of steps — **$O(\log n)$** instead of $O(n)$.



Recall Chapter 2: B-trees are designed to minimize **disk block reads**, which is exactly why they suit databases. (A **hash index** is an alternative — $O(1)$ for exact-match lookups, but it can't do range queries or sorting, so B-trees are the general default.)

12.4 What indexes speed up

- **Filtering** on an indexed column (`WHERE user_id = 42`).
- **JOINS** on indexed keys (foreign keys are prime candidates).
- **Sorting** (`ORDER BY`) — the index is already sorted.
- **Range queries** (`WHERE created_at > ...`) — a B-tree strength.

The **primary key** is automatically indexed. You typically add indexes on foreign keys and on columns you frequently filter or sort by.

12.5 The cost of indexes

INDEXES ARE NOT FREE

Every index consumes **storage**, and — more importantly — every `INSERT`, `UPDATE`, and `DELETE` must **also update every index** on that table. So indexes speed up reads but **slow down writes**. On a write-heavy table, too many indexes hurt. The rule: index the columns you actually query, not every column “just in case.” Indexing is a classic read-vs-write trade-off.

12.6 Types of indexes

Type	What it is
Clustered	The table's rows are physically stored in the index's sorted order. One per table (often the primary key). Range scans on it are very fast.
Non-clustered	A separate structure that points to the rows. You can have many.
Composite (multi-column)	Indexes several columns together. Order matters — an index on <code>(a, b)</code> helps queries filtering by <code>a</code> or by <code>a</code> then <code>b</code> , but not by <code>b</code> alone (the “leftmost prefix” rule).

Covering

An index that contains all columns a query needs, so the database answers it from the index alone — never touching the table.

12.7 Reading a query plan

How do you know whether your index is even being used? Ask the database. The `EXPLAIN` command shows the **query plan** — the steps the database will take, including whether it does a slow *sequential/full scan* or a fast *index scan/seek*.

```
EXPLAIN SELECT * FROM posts WHERE user_id = 42;
-- look for: "Index Scan using posts_user_id_idx"    (good)
-- vs:      "Seq Scan on posts"                    (bad on a big table)
```

KEY INSIGHT — MEASURE, DON'T GUESS

Adding an index doesn't guarantee the database will use it, and the reason a query is slow is often surprising. `EXPLAIN` turns performance from guesswork into evidence: run it, see whether your index is used, and confirm the fix actually changed the plan. This one habit prevents most "I added an index but it's still slow" confusion.

12.8 Query performance beyond indexes

- **The N+1 problem:** fetching a list, then firing one more query *per item* in a loop — 1 + N queries where a single JOIN or batched query would do. A top cause of slow apps; fix by loading related data in one query.
- **Select only what you need:** avoid `SELECT *`; fetching unused columns wastes I/O and blocks covering indexes.
- **Paginate efficiently:** huge `OFFSET` values get slow; prefer **keyset/cursor** pagination (recall the feed cursor from earlier design work).
- **Reuse connections:** connection pooling (Chapter 3) avoids per-query setup cost.
- **Denormalize for reads** where justified (Chapter 13) — trading some duplication for fewer JOINs.

Common mistakes

WATCH OUT

- No index on columns you filter or JOIN on — silent full table scans as data grows.
- **Over-indexing** — adding indexes everywhere and crippling write performance.
- Assuming an index is used without checking `EXPLAIN`.
- Getting composite-index column order wrong (violating the leftmost-prefix rule).
- The **N+1** query pattern hidden inside an innocent-looking loop.
- `SELECT *` and giant `OFFSET` pagination on large tables.

Interview questions

1. Why is a full table scan slow, and how does an index fix it?

2. How does a B-tree index give $O(\log n)$ lookups? What can't a hash index do?
3. What do indexes cost? When can adding an index hurt?
4. Explain clustered vs non-clustered, and what a covering index is.
5. What is the leftmost-prefix rule for composite indexes?
6. How do you tell whether a query uses an index?
7. What is the N+1 query problem, and how do you fix it?

Hands-on exercises

1. On a table with a million rows, run a filtered query with and without an index and compare the time.
2. Use `EXPLAIN` on a query and identify whether it does a sequential scan or an index scan.
3. Design a composite index for a query that filters by `status` and sorts by `created_at`.
4. Spot the N+1 pattern in a snippet that lists posts and then loads each post's author in a loop, and rewrite it.
5. Decide which three columns of an `orders` table you'd index, and justify each.

Mini project

MAKE A SLOW QUERY FAST

Load a table with a few hundred thousand rows. Pick a query that filters on a non-key column and time it — then run `EXPLAIN` and confirm it's a full scan. Add an index on that column, re-run `EXPLAIN` (confirm it now uses the index), and time it again. Record the before/after numbers. Then add five more indexes and time a batch of `INSERT` s to feel the write-side cost. You'll have measured both sides of the indexing trade-off yourself.

Large project (capstone continues)

YOUR SOCIAL PLATFORM — THE INDEXING PLAN

For the schema you designed in Chapter 11, list the **queries** your platform runs most (a user's feed, a post's comments, a user's followers) and design the **indexes** that make each fast — including at least one **composite** index (e.g. `(user_id, created_at)` for a user's posts newest-first). Note the write cost on your highest-volume tables, and flag any query at risk of the N+1 problem. This tuning is what lets a single database survive your read load before you add replicas (Chapter 16).

Chapter summary

- Without an index, a filtered query does a **full table scan** ($O(n)$) — fine when small, ruinous at scale.
- An **index** is a sorted lookup structure; a **B-tree** gives $O(\log n)$ lookups and supports ranges and sorting.

- Indexes speed up filtering, JOINS, sorting, and range queries — but **cost storage and slow down writes**, so index what you query, not everything.
- Know **clustered, composite (leftmost-prefix), and covering** indexes.
- Use **EXPLAIN** to see the query plan — measure whether the index is used rather than guessing.
- Beyond indexes, fix the **N+1 problem**, avoid `SELECT *`, paginate with cursors, and reuse connections.

COMING NEXT

Chapter 13 — Normalization, Denormalization & Data Modeling. How should you actually shape your tables? We'll cover normalization (eliminate duplication for integrity) and its deliberate opposite, denormalization (add duplication for read speed) — and when each is the right trade-off.